

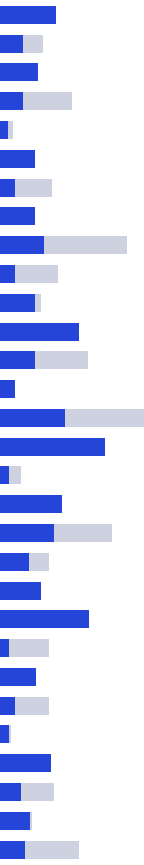
2026 Spring

DCLab Final

Magnetic Field Finger-Tracking Musical Glove

Group 7

電機四 B11202010 韓裕民、B11901188 吳毅恩、B11901193 劉又慈

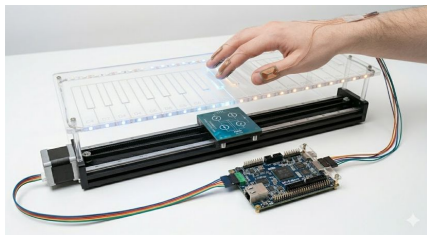


Outline

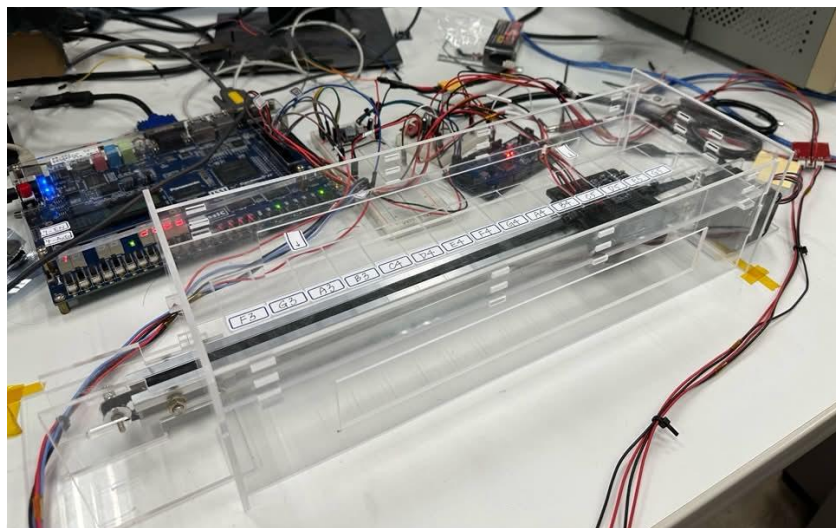
- Hardware Implementation
 - FPGA System Architecture
 - Filtering Strategy
 - Key Classification
- 

Project Goal

Build a **magnetic-field-based piano interface** that detects finger/key position and converts it into musical notes.

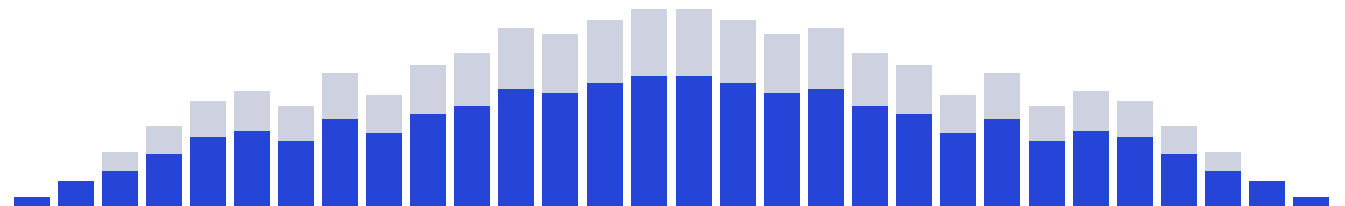


Initial proposal





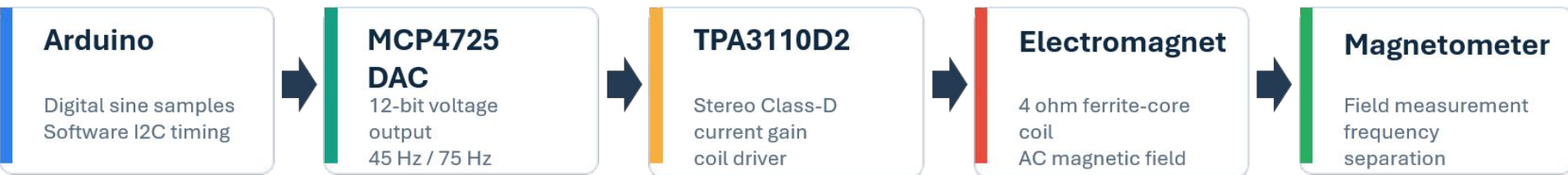
Hardware Implementation





Hardware Architecture

Arduino-controlled DAC waveform generation and Class-D coil driving for dual-frequency electromagnetic localization.



- Arduino sends discrete DAC codes over two Software I2C buses.
- DAC outputs low-power analog sine waveforms.
- Power amplifier converts voltage waveform into coil current.
- Magnetometer observes the resulting AC magnetic field.





Electromagnet design

- Target coil impedance: around 4 Ω
- More turns increase magnetomotive force for a given current.
- Resistance, heating, and amplifier loading must be balanced.



Coil condition	System effect	Design implication
Too low	Large current, heat, protection risk	May overload the TPA3110D2 or power supply
Too high	Low current and weaker magnetic field	May reduce magnetometer signal amplitude
Around 4 Ω	Matches common low-Z audio load region	Practical current while staying near datasheet load range





DAC – MCP4725

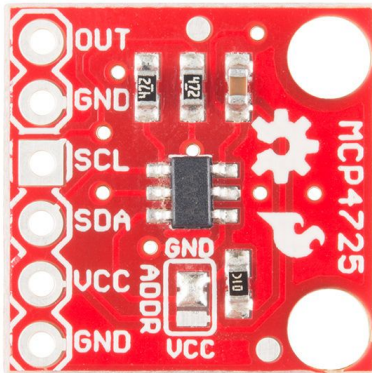
MCP4725 role

Single-channel DAC that turns Arduino sample codes into a low-power analog sine control voltage.

Why DAC instead of PWM only?

Clean low-frequency analog input for the power amplifier stage.

Datasheet item	Value used in this design
Resolution	12-bit DAC code range, centered around code 2048
Interface	I2C serial interface
Supply / reference	2.7-5.5 V, VREF driven from VDD
Output	Rail-to-rail voltage output, small-signal load only
I2C speed	100 kbit/s, 400 kbit/s, up to 3.4 Mbit/s HS mode







Dual-Frequency Magnetic Field Encoding

Channel 1

45 Hz sine -> Coil 1 -> magnetic field component at 45 Hz

Channel 2

75 Hz sine -> Coil 2 -> magnetic field component at 75 Hz



Magnetometer signal

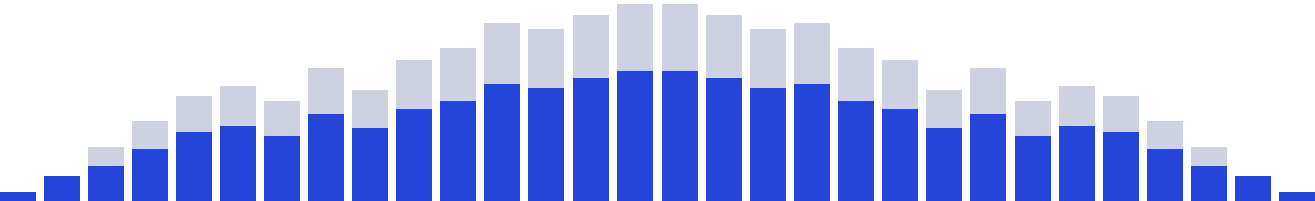
Measured field contains a mixture of environmental field plus 45 Hz and 75 Hz AC components.



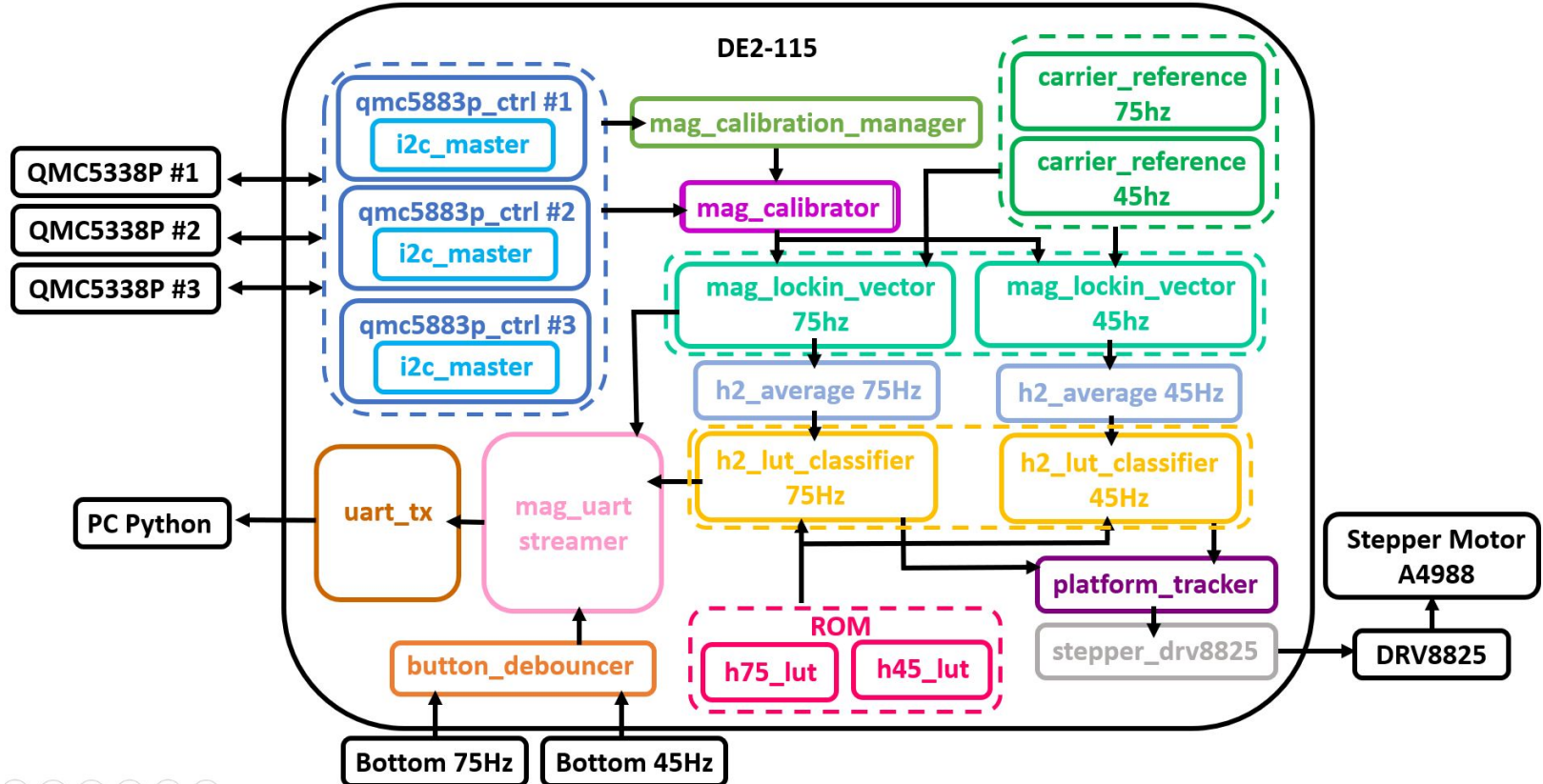


FPGA

System Architecture

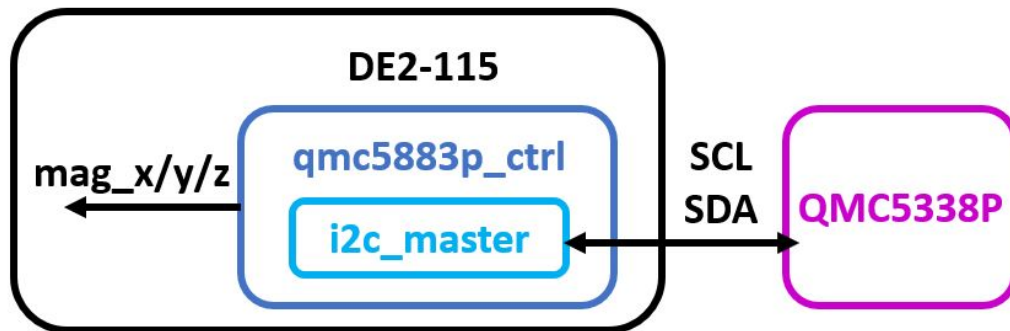


System Overview



Module: qmc5883l_ctrl

Controls QMC5883P magnetometer through I2C



Chip ID Check

Verify sensor identity
read `0x00`
expect `0x80`

Control Register 1 Setup

Continuous mode
+ 200 Hz ODR

Control Register 2 Setup

$\pm 2\text{G}$ magnetic range +
set/reset enabled

Magnetic Data Readout

Read data then
combine into
`mag_x/y/z`



Module: mag_calibration_manager

Generates offset and scale coefficients for each active magnetometer axis before real-time magnetic-field processing

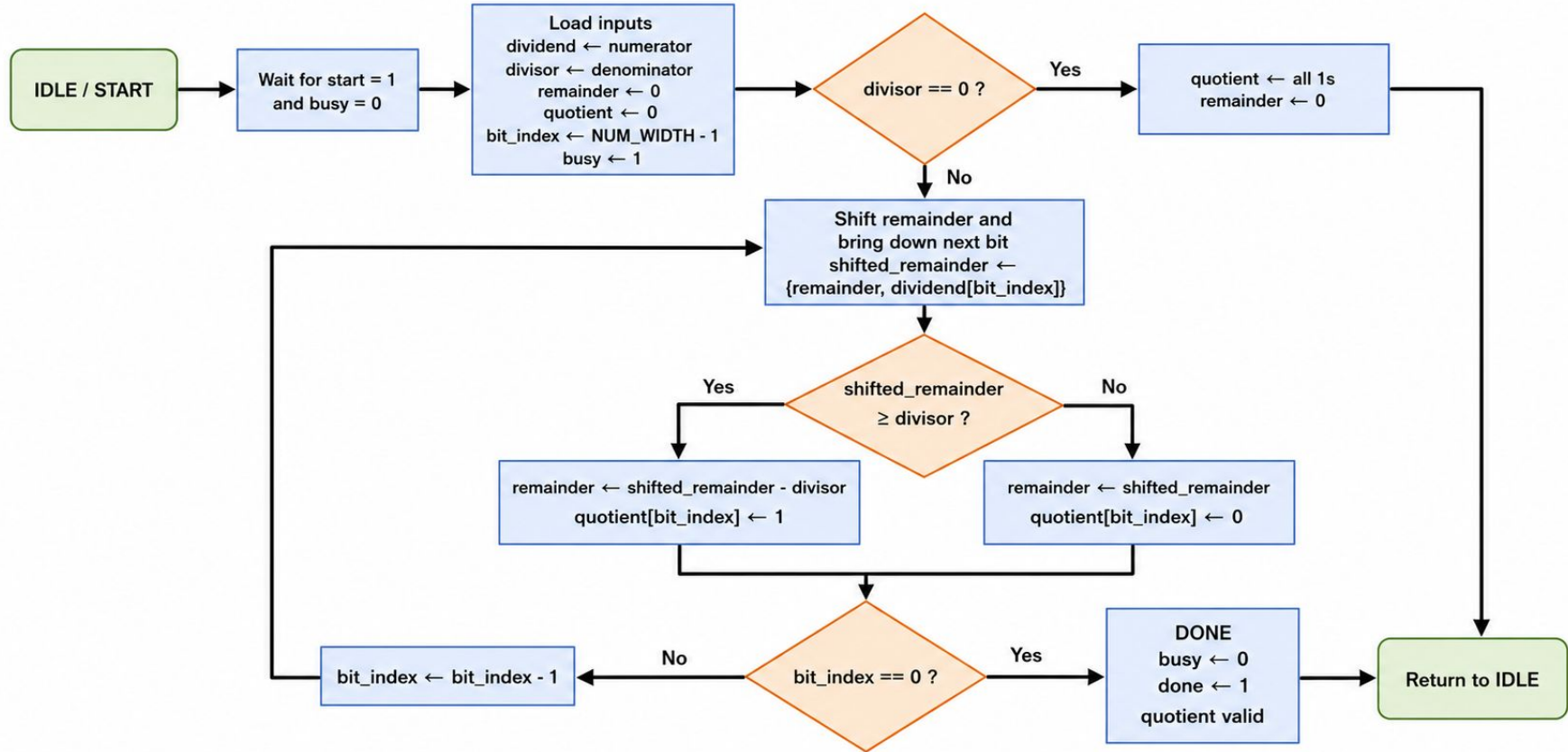


Module: mag_calibrator

Combinational datapath that continuously produces outputs whenever the raw input or calibration coefficients change

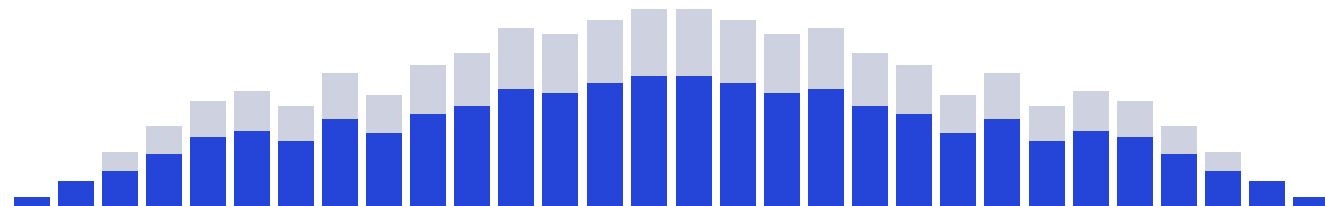
$$\text{corrected} = (\text{raw} - \text{offset}) \times \text{scale}$$

Module: unsigned_divider





Filtering Strategy





Core Idea

The calibrated magnetic signal is multiplied by a 75/45 Hz sine/cosine reference and accumulated over a window

- 75/45 Hz component accumulates coherently
- DC component, noise and other frequencies tend to cancel
- Output represents the detected 75/45 Hz magnetic-field strength

`carrier_reference_75hz`: generates 75/45 Hz sine/cosine reference

`mag_lockin_vector_75hz`: performs lock-in filtering and energy computation



Module: carrier_reference_75hz

Generates digital sine and cosine reference signals

+1 → 32767
-1 → -32768
+0.707 → 23170 ($0.7071 \times 32768 \approx 23170$)
-0.707 → -23170

```
always @(*) begin
  case (phase_index)
    3'd0: begin sine_q15 = 16'sd0;      cosine_q15 = 16'sd32767; end
    3'd1: begin sine_q15 = 16'sd23170; cosine_q15 = 16'sd23170; end
    3'd2: begin sine_q15 = 16'sd32767; cosine_q15 = 16'sd0;      end
    3'd3: begin sine_q15 = 16'sd23170; cosine_q15 = -16'sd23170; end
    3'd4: begin sine_q15 = 16'sd0;      cosine_q15 = 16'sh8000;  end
    3'd5: begin sine_q15 = -16'sd23170; cosine_q15 = -16'sd23170; end
    3'd6: begin sine_q15 = 16'sh8000;  cosine_q15 = 16'sd0;      end
    default: begin
      sine_q15 = -16'sd23170;
      cosine_q15 = 16'sd23170;
    end
  endcase
end
```

phase_index	Angle	sine	cosine
0	0°	0	+1
1	45°	+0.707	+0.707
2	90°	+1	0
3	135°	+0.707	-0.707
4	180°	0	-1
5	225°	-0.707	-0.707
6	270°	-1	0
7	315°	-0.707	+0.707

$75 / 200 = 0.375 \text{ cycle} = 3/8 \text{ cycle}$ → 0 → 3 → 6 → 1 → 4 → 7 → 2 → 5 → 0 ...

$45 / 200 = 0.225 \text{ cycle} = 9/40 \text{ cycle}$

Module: mag_lockin_vector_75hz

Extracts the 75 Hz magnetic-field component from the calibrated X/Y/Z data

Multiply with 75 Hz Reference

$X_I = X \times \cos_ref$	$X_Q = X \times \sin_ref$
$Y_I = Y \times \cos_ref$	$Y_Q = Y \times \sin_ref$
$Z_I = Z \times \cos_ref$	$Z_Q = Z \times \sin_ref$

both sin and cos
insensitive to phase difference



Accumulate over 100 Samples

The module sums these I/Q products over a 100-sample rolling window:

$I_acc = \text{sum}(\text{signal} \times \cos_ref)$
 $Q_acc = \text{sum}(\text{signal} \times \sin_ref)$

$acc_new = acc_old + new_product - old_product$

Module: mag_lockin_vector_75hz

Square and sum

X_I_acc, X_Q_acc
Y_I_acc, Y_Q_acc
Z_I_acc, Z_Q_acc

output

$$\longrightarrow X_I^2 + X_Q^2 + Y_I^2 + Y_Q^2 + Z_I^2 + Z_Q^2$$



Why It Has a Filtering Effect

75 Hz component → accumulates

DC / noise / other frequencies → alternate between positive and negative values → cancel out

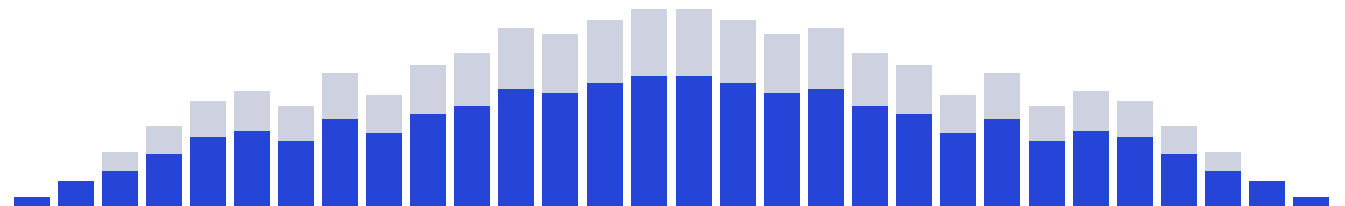
Where to store 100 Samples?

M9K block RAM

```
(* ramstyle = "M9K" *) reg signed [31:0] x_i_window [0:WINDOW_SAMPLES-1];
(* ramstyle = "M9K" *) reg signed [31:0] x_q_window [0:WINDOW_SAMPLES-1];
(* ramstyle = "M9K" *) reg signed [31:0] y_i_window [0:WINDOW_SAMPLES-1];
(* ramstyle = "M9K" *) reg signed [31:0] y_q_window [0:WINDOW_SAMPLES-1];
(* ramstyle = "M9K" *) reg signed [31:0] z_i_window [0:WINDOW_SAMPLES-1];
(* ramstyle = "M9K" *) reg signed [31:0] z_q_window [0:WINDOW_SAMPLES-1];
```



Key Classification



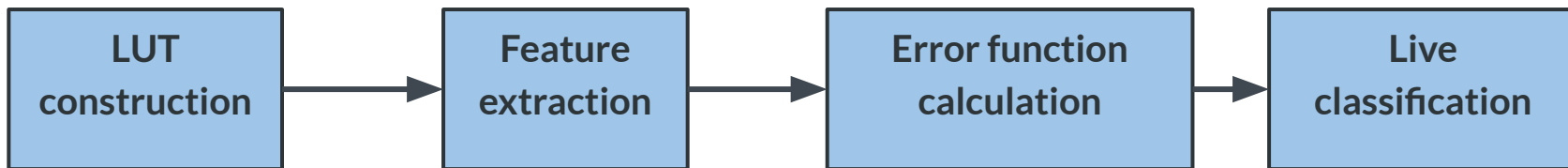


LUT-Based Key Classification

Main idea:

Each key position produces a unique H^2 pattern across the sensors.

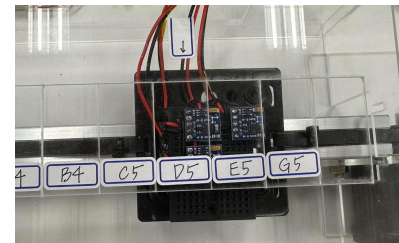
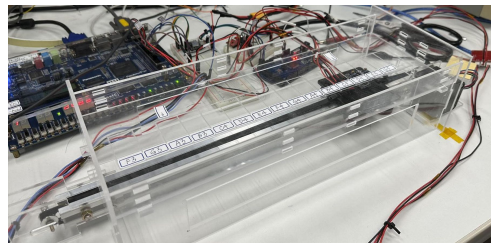
We record these patterns into a LUT, then FPGA compares live data against the LUT.





LUT Construction

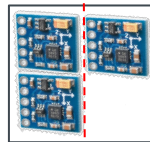
- 15 piano keys.
- 3 sensors (magnetometers).



$$\text{global_index} = \text{platform_center_global} + \text{local_index}$$

Key's
global index

					Local index									
					-2	-1	0	1	2					
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14



sensors

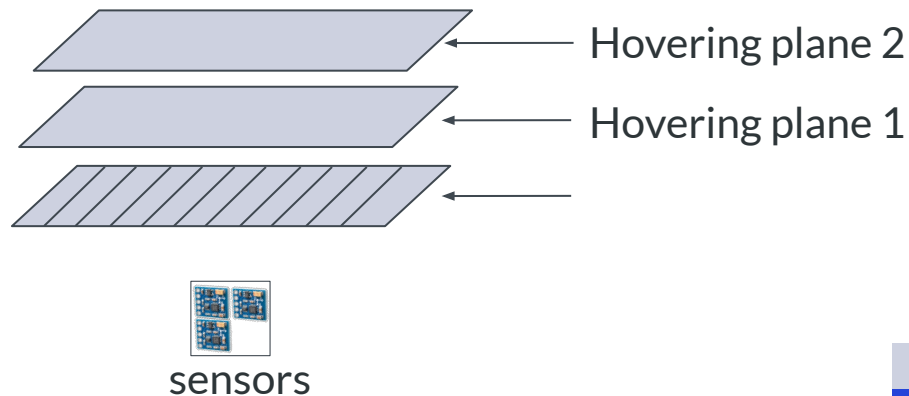
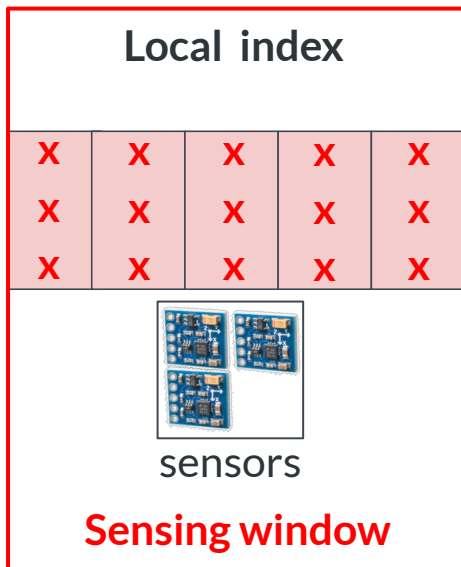
Sensing window





LUT Construction

- Three sampling points(**x**) per key, total 15 sampling points per z plane.
- Since fingers may not always touch the key plane, we also construct LUTs for two slightly elevated z-planes to improve classification robustness.





Feature Extraction

- Instead of directly comparing raw H^2 , normalize each sensor:

$$\begin{aligned}\text{total} &= H1^2 + H2^2 + H3^2 \\ f1 &= H1^2 / \text{total} \\ f2 &= H2^2 / \text{total} \\ f3 &= H3^2 / \text{total}\end{aligned}$$

- So each LUT entry stores:

key_id	f1	f2	f3	ln(total)
--------	----	----	----	-----------





Why Normalize H^2 ?

- Raw H^2 compares “how strong the signal is.”
- Normalized H^2 compares “which sensor is strongest relative to the others.”
- Position is better represented by the relative sensor pattern.
- Normalization removes scale.
- Final error keeps a small strength term.





Error Function / Live Classification

$$\text{ratio error} = (f_{1_{live}} - f_{1_{LUT}})^2 + (f_{2_{live}} - f_{2_{LUT}})^2 + (f_{3_{live}} - f_{3_{LUT}})^2$$

$$\text{strength error} = (\ln total_{live} - \ln total_{LUT})^2$$

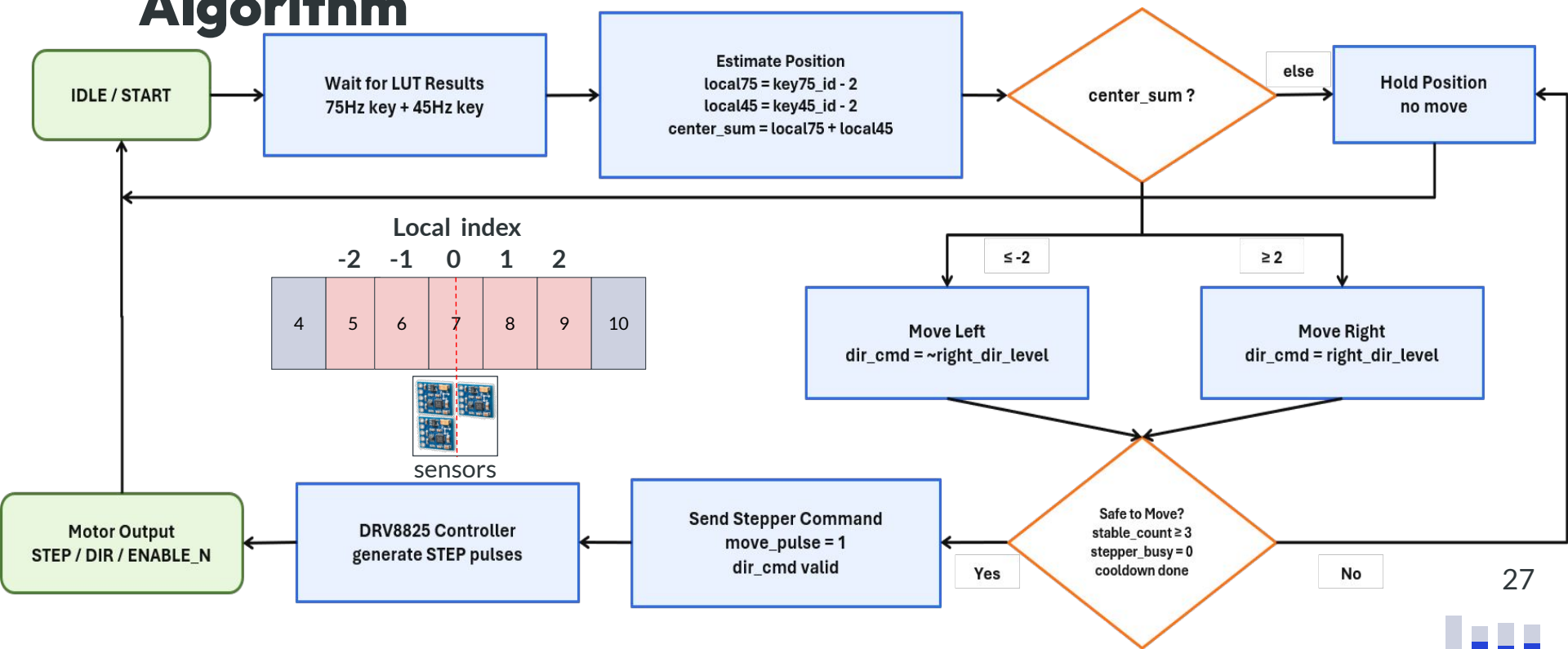
$$\text{final error} = \text{ratio error} + 0.05 \times \text{strength error}$$

Predicted key = key with minimum final error





Stepper Motor Auto-Tracking Control Algorithm





Thanks!